

MEL Editor

User Manual

Art2Dec SoftLab · mshell Ecosystem · GPL3

v1.3.1 · Igor Lukyanov · igor.lukyanov@appservgrid.com

Table of Contents

1. Introduction
2. Main Features
3. Supported Platforms
4. Supported Languages & Syntax Highlighting
5. Installation Guide
6. Using MEL Editor
7. Command-Line Options
8. Multiple CLI Options
9. Content Loading Methods
10. Keybindings
11. UTF-8 Support
12. Backup Mechanism
13. Integration with LLM Models
14. Uninstallation
15. FAQ and Troubleshooting
16. License and Contact Information

1. Introduction

MEL (Mini Embedded Light) Editor is a terminal-based, minimalistic text editor for Linux and Unix platforms, written in pure C from scratch. Designed for low-resource environments: Kubernetes Pods, Docker Containers, IoT Devices (ARM64), Cloud Platforms (AWS, GCP, OCI).

MEL v1.3.1 includes full UTF-8 support, Lua and Rust syntax highlighting, full-screen LLM prompt editor with mshell and curl integrations, system clipboard support, and a dependency installer script. It does not use curses or ncurses.

Distributed under the GNU GPL3 license.

2. Main Features

- Lightweight & Fast: no curses dependency, efficient in constrained environments.
 - Cross-Platform: Linux x86_64, macOS Intel & Apple Silicon, Raspberry Pi ARM64.
 - Full UTF-8 Support: correct display and editing of Russian, Japanese, CJK, Emoji.
 - Syntax Highlighting: 15 languages including Lua and Rust.
 - Backup Mechanism: creates .bak on first save with -b flag.
 - Ctrl-W: LLM via mshell (ollama1/2/3) — full-screen prompt editor, Ctrl-O to send.
 - Ctrl-K: LLM via curl + ollama.conf — local or remote models, Ctrl-O to send.
 - Ctrl-T: Paste from system clipboard (xclip/xsel/pbpaste).
 - Dependency Installer: install_deps.sh auto-detects platform and installs required libraries.
-

3. Supported Platforms

Platform	Architecture	Status
Ubuntu 22.04 / 24.04 LTS	x86_64	Tested ✓
Debian 12 (Bookworm)	ARM64 (Raspberry Pi 4/5)	Tested ✓
Debian 13 (Trixie)	ARM64 (Raspberry Pi 4/5)	Tested ✓
macOS Sequoia (Intel)	x86_64	Tested ✓
macOS Sequoia (Apple Silicon)	M1/M2/M3/M4 arm64	Tested ✓
Docker / Kubernetes Pods	x86_64 / ARM64	Compatible
AWS / GCP / OCI cloud	x86_64 / ARM64	Compatible

■ On macOS, only Ctrl-K (curl mode) is available for LLM integration. mshell is not yet available for macOS.

4. Supported Languages & Syntax Highlighting

MEL v1.3.1 supports syntax highlighting for 15 languages:

Language	File Extensions
C / C++	.c .h .cpp .hpp .cc .cxx
Java	.java
Python	.py .pyc .pyo .pyw

PHP	.php .phtml
Bash	.sh
Mshell	.ms
JavaScript	.js .jsx
JSON	.json .jsonp
XML	.xml (partial)
SQL	.sql
Ruby	.rb
Go	.go
Lua	.lua
Rust	.rs

■ Lua: -- for single-line, --[[]] for multi-line. Rust: // and /* */.

5. Installation Guide

5.1 Install Dependencies (Recommended First Step)

MEL requires libcurl and libjson-c. Use the provided script — auto-detects platform:

```
bash install_deps.sh
```

Platforms: Ubuntu 22.04/24.04, Debian 12/13 ARM64, macOS Intel, macOS Apple Silicon (M1/M2/M3/M4).

■ On macOS the script installs Homebrew if absent, then installs via brew.

5.2 Build from Source

```
git clone https://github.com/igor101964/mel.git
cd mel/
bash install_deps.sh
make
```

5.3 Install System-Wide

```
make install # installs to /usr/local/bin/mel
```

5.4 Pre-Built Binaries

Binary	Platform
mel-linux-x86_64	Ubuntu 22.04/24.04 LTS x86_64
mel-linux-arm64	Debian 12/13 ARM64 (Raspberry Pi 4/5)
mel-macos-intel	macOS Sequoia Intel x86_64

```
sudo cp mel-linux-x86_64 /usr/local/bin/mel
sudo chmod +x /usr/local/bin/mel
```

5.5 Makefile Targets

Target	Action
--------	--------

make	Build with -O2 optimization
make debug	Build with debug symbols (-g)
make install	Install to /usr/local/bin/mel
make uninstall	Remove from /usr/local/bin, /usr/bin, /bin
make clean	Remove compiled binary

6. Using MEL Editor

Open a file:

```
mel myfile.py
```

Start without a file:

```
mel
```

Open at specific line:

```
mel -l 42 myfile.c
```

7. Command-Line Options

Option	Action
mel -h --help	Print help and keybindings
mel -v --version	Print version
mel -b <file>;	Create .bak backup before first save
mel -l <N>; <file>;	Open file at line N
mel -w <cols>; <file>;	Set visual column width marker

8. Multiple CLI Options

Options can be combined:

```
mel -b -l 10 myfile.txt
```

Creates a .bak backup and opens myfile.txt at line 10.

9. Content Loading Methods

9.1 Open a File

```
mel myfile.py
```

9.2 Pipe Data to MEL

```
cat myfile.txt | mel
```

9.3 Multiple Commands

```
(df -m ; ls -la) | mel
```

9.4 grep Filter

```
(df -m | grep /boot) | mel
```

10. Keybindings

Keybinding	Action
Ctrl-Q	Exit (press 3 times if unsaved)
Ctrl-S	Save (prompts for filename if new)
Ctrl-Z	Undo
Ctrl-Y	Redo
Ctrl-C	Copy line
Ctrl-X	Cut line
Ctrl-V	Paste line
Ctrl-E	Flip line upwards
Ctrl-D	Flip line downwards
Ctrl-B	Hide / Show line numbering
Ctrl-F	Search by pattern
Ctrl-N	Forward search (after Ctrl-F)
Ctrl-R	Backward search (after Ctrl-F)
Ctrl-J	Global find & replace
Ctrl-G	Go to line number
Ctrl-Left	Move cursor one word left
Ctrl-Right	Move cursor one word right
Ctrl-P	Pause mel (type "fg" to resume)
Ctrl-T	Paste from system clipboard (xclip/xsel/pbpaste)
Ctrl-W	LLM via mshell (ollama1/2/3) — full prompt editor, Ctrl-O=send, ESC=cancel
Ctrl-K	LLM via curl (ollama.conf) — full prompt editor, Ctrl-O=send, ESC=cancel
Ctrl-H	Toggle help screen
Home	Beginning of line
End	End of line
PgUp / PgDn	Page up / Page down
Backspace	Delete character (UTF-8 aware)
Arrow keys	Move cursor (UTF-8 aware — moves by full character)

11. UTF-8 Support

MEL v1.3.1 includes full UTF-8 support. The following operations are UTF-8 aware:

- Display: multi-byte characters (Russian, Japanese, CJK, Emoji) render correctly.
- Cursor navigation: Left/Right arrows move by full Unicode character, not by byte.
- Backspace: deletes the complete preceding Unicode character (1–4 bytes).

- Insert: typing any Unicode character inserts all bytes correctly.
- Syntax highlighting: continuation bytes coloured consistently with their lead byte.
- LLM Prompt Editor: full UTF-8 support for entering prompts in any language.

■ Terminal must be set to UTF-8 (standard on all modern Linux/macOS terminals).

Quick test:

```
echo '■■■■■■■ ■' > test.txt && mel test.txt
```

12. Backup Mechanism

Use `-b` flag to create `.bak` backup on first save:

```
mel -b myfile.c # creates myfile.c.bak before saving
```

13. Integration with LLM Models

MEL v1.3.1 provides two independent LLM integration methods, each using a full-screen prompt editor.

In prompt mode: write your prompt using the full MEL editor (UTF-8, multi-line, any length), then press **Ctrl-O** to send or **ESC** to cancel. The response is inserted at the cursor position in the original file.

13.1 Ctrl-W — mshell Integration (Linux only)

Uses mshell built-in commands `ollama1`, `ollama2`, `ollama3` configured in `~/.mshellrc`. Supports multiple models configured in the mshell Ecosystem.

- Ctrl-W opens the full-screen prompt editor.
- Enter model number (1, 2, or 3) at the prompt.
- Write your prompt — any length, any language including UTF-8.
- Ctrl-O sends the prompt via mshell `ollama1/2/3`.
- Response is inserted at cursor position.

■ Ctrl-W requires mshell to be installed and ollama models configured in `~/.mshellrc`. Available on Linux only.

13.2 Ctrl-K — curl Integration (Linux & macOS)

Uses libcurl to call any Ollama-compatible REST API — local or remote. Configured via `~/.config/mel/ollama.conf`.

- Ctrl-K opens the full-screen prompt editor.
- Write your prompt — any length, any language including UTF-8.
- Ctrl-O sends the prompt via HTTP POST to the configured endpoint.
- Response is inserted at cursor position.

Configuration: `~/.config/mel/ollama.conf`

```
OLLAMA1_API_URL=http://localhost:11434/api/generate
OLLAMA1_MODEL=llama3
```

The API URL can point to any Ollama-compatible server — local or remote on the network:

```
# Remote model on local network:
OLLAMA1_API_URL=http://192.168.1.100:11434/api/generate
OLLAMA1_MODEL=qwen:7b
```

■ Both OLLAMA1_API_URL and OLLAMA1_MODEL must be present. On macOS use only Ctrl-K — mshell is not yet available for macOS.

Requirements for Ctrl-K

- Ollama running: `ollama serve`
- Model pulled: `ollama pull llama3`
- libcurl and libjson-c installed (handled by `install_deps.sh`)

13.3 Ctrl-T — System Clipboard Paste

Pastes content from the system clipboard directly into the editor at cursor position.

Uses xclip (Linux) or xsel (Linux) or pbpaste (macOS) — installed automatically by `install_deps.sh`.

14. Uninstallation

```
make uninstall

# Or manually:
sudo rm -f /usr/local/bin/mel
sudo rm -f /usr/bin/mel
sudo rm -f /bin/mel
```

15. FAQ and Troubleshooting

Q: Unicode text shows as diamonds

A: Ensure terminal is UTF-8. Linux: `export LANG=en_US.UTF-8`. macOS: Terminal → Preferences → Encoding → UTF-8.

Q: make fails: curl/curl.h not found

A: Run: `bash install_deps.sh`

Q: make fails on macOS

A: Run: `bash install_deps.sh` — sets up Homebrew with correct paths for Intel and Apple Silicon.

Q: Ctrl-W shows 'popen failed'

A: Ensure mshell is installed and in PATH. Ctrl-W is available on Linux only.

Q: Ctrl-K: 'Invalid Ollama config'

A: Create `~/.config/mel/ollama.conf` with OLLAMA1_API_URL and OLLAMA1_MODEL fields. See section 13.2.

Q: Ctrl-K: no response

A: Ensure Ollama is running: `ollama serve`. Check model: `ollama list`. Verify API URL is reachable.

Q: Ctrl-T: no paste

A: Install xclip: `sudo apt install xclip` (Linux) or use pbpaste on macOS (built-in).

Q: Lua/.lua or Rust/.rs not highlighted

A: Use mel v1.3.1. Rebuild: `make clean && make`.

16. License and Contact Information

MEL is distributed under the GNU General Public License v3 (GPL3).

Field	Info
Version	1.3.1
License	GPL3
Author	Igor Lukyanov
Contact	igor.lukyanov@appservgrid.com
Repository	https://github.com/igor101964/mel
Organization	Art2Dec SoftLab
Ecosystem	mshell Ecosystem